

```

classDiagram
    package com.gwj.service {
        class UsuarioService {
            +UsuarioService()
        }
        class GenericService {
            #repository: IRepository<T>
            +GenericService(entityClass: Class)
            +GenericService(repository: IRepository<T>)
        }
        class ServiceRegistry {
            -registry: Map<String, IService>
            +getService(entityName: String): IService<T>
        }
        class IService {
            <<interface>>
            +create(entity: T): T
            +read(entity: T): List<T>
            +update(entity: T): Long
            +delete(entity: T): Long
        }
    }

    package com.gwj.controller {
        class LoginController {
            +showLoginForm(session: HttpSession): String
            +processLogin(email: String, senha: String, request: HttpServletRequest, model: Model): String
            +logout(request: HttpServletRequest): String
        }
        class Router {
            +home(model: Model): String
            +servico(model: Model): String
            +shop(model: Model): String
            +singleProduct(request: HttpServletRequest, model: Model): String
        }
        class JsonController {
            +create(entityName: String, request: HttpServletRequest): ResponseEntity
            +read(entityName: String, request: HttpServletRequest): ResponseEntity
            +update(request: HttpServletRequest): ResponseEntity
            +delete(entityName: String, request: HttpServletRequest): ResponseEntity
        }
        class GenericViewController {
            +create(entity: String, request: HttpServletRequest, model: Model): String
            +listar(entity: String, request: HttpServletRequest, model: Model): String
            +detalhe(entity: String, request: HttpServletRequest, model: Model): String
            +editar(entity: String, request: HttpServletRequest, model: Model): String
        }
    }

    package com.gwj.model.repository {
        class UsuarioRepository {
            +UsuarioRepository()
        }
        class GenericRepository {
            -entityClass: Class<T>
            +GenericRepository(entityClass: Class<T>)
        }
        class IRepository {
            <<interface>>
            +save(entity: T): T
            +query(entity: T): List<T>
            +update(entity: T): Long
            +delete(entity: T): Long
        }
    }

    package com.gwj.model.domain.factory {
        class SchemaValidator {
            +validateAllEntities()
        }
        class SimpleObjectFactory {
            +create(name: String): IEntity
        }
    }

    package com.gwj.model.dataTransferObject {
        class EntityMapper {
            +fillEntity(entity: IEntity, request: HttpServletRequest): IEntity
        }
    }

    package com.gwj.model.dataAccessObject {
        class QueryBuilder {
            +select(fields: String): QueryBuilder
            +from(table: String): QueryBuilder
            +where(condition: String): QueryBuilder
            +insertInto(table: String): QueryBuilder
            +update(table: String): QueryBuilder
            +build(): String
        }
        class IMapper {
            +getEntityHierarchy(startClass: Class): List<Class>
            +getTableName(clazz: Class): String
            +getColumnNome(clazz: Class, method: Method, isEntityValue: boolean): String
            +convertToTargetType(value: Object, targetType: Class, field: Field): Object
            +convertPascalCaseToSnakeCase(name: String): String
            +capitalize(str: String): String
        }
        class ConnectionDB {
            -instance: ConnectionDB
            +getConnection(): Connection
            +getInstance(): ConnectionDB
        }
    }

    package com.gwj.service.transaction {
        class UnitOfWork {
            -currentConnection: ThreadLocal<Connection>
            -inTransaction: ThreadLocal<Boolean>
            +getConnection(): Connection
            +beginTransaction()
            +commit()
            +rollback()
            +close()
        }
    }

    UsuarioService --|> GenericService
    GenericService ..> UsuarioRepository : usa
    GenericService ..> ServiceRegistry : usa
    GenericService ..> IService : delega para
    ServiceRegistry o--> IService : gerencia
    LoginController ..> ServiceRegistry : usa
    Router ..> ServiceRegistry : usa
    JsonController ..> ServiceRegistry : usa
    GenericViewController ..> ServiceRegistry : usa
    GenericRepository --|> IRepository
    GenericRepository ..> IRepository : constrói SQL
    SimpleObjectFactory ..> IService : cria instâncias
    EntityMapper ..> IService : popula
    IMapper ..> QueryBuilder : realiza mapeamento
    IMapper ..> IMapper : obtém Connection
    IMapper ..> ConnectionDB : opcional
    UnitOfWork ..> ConnectionDB : obtém Connection

```

